K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

School of Engineering & Technology

OPERATING SYSTEM

LAB ASSINGMENT 1 - REPORT

Program: BCA (AI & DS)

Course Code – ENCA252 (2026-2027)

Submitted By

RAKESH G

2401201064

BCA (AI & DS)

Submitted To

Jyoti Yadav

1. Introduction

CPU Scheduling is a fundamental function of an operating system that determines the order in which processes are allocated CPU time. Efficient scheduling directly impacts system performance, throughput, and overall user experience.
This report presents the implementation and analysis of two widely used CPU scheduling algorithms:

- First Come First Serve (FCFS): Executes processes in the order of their arrival.
- Shortest Job First (SJF - Non-Preemptive): Selects the process with the smallest burst time among available processes.

2. Learning Objectives

- Understand CPU scheduling concepts and process attributes
- Implement FCFS and SJF scheduling algorithms using Python
- Calculate Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT)
- Visualize execution using Gantt charts
- Analyze and compare performance of scheduling algorithms

3. Tools & Technology

- Programming Language: Python 3.x
- Libraries Used: os, copy (built-in)
- IDE: Visual Studio Code
- Platform: Windows 11

Task 1 — Process Creation & Input Handling

Description

A Process class was created with attributes:

- Process ID (PID)
- Arrival Time (AT)
- Burst Time (BT)

Additional parameters such as Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) were also included for computation. Processes were taken as input from the user and stored in a list for further scheduling.

Process Input Table

PID	Arrival Time (AT)	Burst Time (BT)
P1	0	6
P2	2	8
P3	4	7
P4	6	3
P5	8	4

Task 2 — FCFS Scheduling

Algorithm Description
First Come First Serve (FCFS) is a non-preemptive scheduling algorithm in which processes are executed in the order of their arrival time. If no process is available, the CPU remains idle until the next process arrives.

Formulas
- Turnaround Time (TAT) = CT - AT
- Waiting Time (WT) = TAT - BT

FCFS Results Table

PID	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	2	8	14	12	4
P3	4	7	21	17	10
P4	6	3	24	18	15
P5	8	4	28	20	16
Average	**-**	**-**	**-**	**14.60**	**9.00**

Task 3 — SJF Scheduling (Non-Preemptive)

Algorithm Description

Shortest Job First (SJF) is a non-preemptive scheduling algorithm that selects the process with the smallest burst time from the ready queue. It reduces average waiting time but may cause starvation for longer processes.

SJF Results Table

PID	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P4	6	3	9	3	0
P5	8	4	13	5	1
P3	4	7	21	17	10
P2	2	8	28	26	18
Average	**-**	**-**	**-**	**11.40**	**5.80**

Task 4 — Gantt Chart Representation
Objective
To visually represent the execution order of processes using Gantt charts.

FCFS Gantt Chart
Execution Order: P1 -> P2 -> P3 -> P4 -> P5

| P1 | P2 | P3 | P4 | P5 |
0 6 14 21 24 28

SJF Gantt Chart
Execution Order: P1 -> P4 -> P5 -> P3 -> P2

| P1 | P4 | P5 | P3 | P2 |
0 6 9 13 21 28

Observation
The Gantt charts clearly show that SJF executes shorter processes earlier, reducing waiting time compared to FCFS.

Task 5 — Performance Analysis & Comparison
Comparison Table

Metric	FCFS	SJF
Avg Turnaround Time	14.60	11.40
Avg Waiting Time	9.00	5.80
Starvation Risk	No	Yes
Complexity	Simple	Moderate

Which Algorithm is Better?
SJF performs better than FCFS in terms of average waiting time and turnaround time.

Justification
- FCFS suffers from the convoy effect, where short processes wait behind long ones.
- SJF minimizes waiting time by executing shorter jobs first.
- Example: Process P4 waited 15 units in FCFS but 0 units in SJF.

Trade-offs
- FCFS: Simple, fair, no starvation, but inefficient
- SJF: Optimal performance, but may cause starvation and requires burst time prediction

Output Screenshots

The following screenshots show the execution of the program for FCFS and SJF scheduling algorithms.

Task 1 — Process input table

```
>>> TASK 1: Process Input

====================================================
         CPU SCHEDULING SIMULATOR
====================================================

Enter number of processes (minimum 4): 5

  Process 1:
    PID        : P1
    Arrival Time: 0
    Burst Time  : 6

  Process 2:
    PID        : P2
    Arrival Time: 2
    Burst Time  : 8

  Process 3:
    PID        : P3
    Arrival Time: 4
    Burst Time  : 7

  Process 4:
    PID        : P4
    Arrival Time: 6
    Burst Time  : 3

  Process 5:
    PID        : P5
    Arrival Time: 8
    Burst Time  : 4


---------------------------------------------
  PROCESS INPUT TABLE
---------------------------------------------
  PID      Arrival Time     Burst Time
---------------------------------------------
  P1       0                6
  P2       2                8
  P3       4                7
  P4       6                3
  P5       8                4
---------------------------------------------
```

Task 2 — FCFS results

```
>>> TASK 2: FCFS Scheduling
=======================================================
  FCFS RESULTS:

  PID      AT     BT     CT     TAT    WT
  ----------------------------------------
  P1       0      6      6      6      0
  P2       2      8      14     12     4
  P3       4      7      21     17     10
  P4       6      3      24     18     15
  P5       8      4      28     20     16
  ----------------------------------------
  Average TAT : 14.60
  Average WT  : 9.00
```

Task 3 — SJF results

```
>>> TASK 3: SJF Scheduling (Non-Preemptive)
=======================================================
  SJF RESULTS:

  PID      AT     BT     CT     TAT    WT
  ----------------------------------------
  P1       0      6      6      6      0
  P4       6      3      9      3      0
  P5       8      4      13     5      1
  P3       4      7      20     16     9
  P2       2      8      28     26     18
  ----------------------------------------
  Average TAT : 11.20
  Average WT  : 5.60
```

Task 4 — Gantt charts

```
>>> TASK 4: Gantt Chart Representation
============================================================

  GANTT CHART — FCFS
  ------------------------------------------------------------
  |  P1  |   P2   |   P3  | P4 | P5 |
  0          6       14     21   24 28
  ------------------------------------------------------------

  GANTT CHART — SJF
  ------------------------------------------------------------
  |  P1  | P4 | P5 |   P3  |   P2   |
  0        6    9     13       20   28
  ------------------------------------------------------------
```

Task 5 — Comparison & analysis

```
>>> TASK 5: Performance Analysis & Comparison

=======================================================
  PERFORMANCE COMPARISON: FCFS vs SJF
=======================================================
  Metric                          FCFS        SJF
  -------------------------------------------
  Average Turnaround Time        14.60      11.20
  Average Waiting Time            9.00       5.60
  -------------------------------------------

  ANALYSIS:
  SJF is BETTER than FCFS.
  SJF reduces Average Waiting Time by 3.40 units.

  WHY?
  SJF always picks the shortest available job, minimizing
  the time longer processes wait. FCFS executes in arrival
. Poor avg WT.
  SJF  : Optimal avg WT, but needs BT prediction &
         can cause starvation of long processes.
=======================================================

  Done! Submit this file + screenshots to LMS.


D:\RAKESH\VSC>
```

Conclusion
This assignment successfully implemented and analyzed FCFS and SJF scheduling algorithms. The results clearly show that SJF provides better performance by minimizing average waiting and turnaround time. However, it has limitations such as starvation of longer processes.

Code Structure Summary

- Process class: Stores process attributes
- get_processes(): Takes user input
- fcfs(): Implements FCFS scheduling
- sjf(): Implements SJF scheduling
- draw_gantt_chart(): Displays Gantt chart
- compare_algorithms(): Compares performance
- main(): Controls execution flow